

GameLab: Mythos Grant

Jens Christian Valen Leynse

INF-3910 Computer Science seminar: GameLab - June 2025



Table of contents

Milestone 1: High concept.....	3
Milestone 2: Design.....	6
Milestone 3: Prototype.....	14
Milestone 4: Alpha	17
Milestone 5: Beta	20
Milestone 6: Live	22

Milestone 1: High concept

Working title
Mythos Grant

Group names and roles
Jens: Developer, designer

The «X»

When scholarly debates hit a dead end, rival historians armed with legendary sword relics settle the argument in a high-stakes tournament—where only one field of research wins the coveted grant money.

Time/Schedule

Milestone	Task	Description	Duration	Start date	End date
Design	Develop design doc	Game and technology defined	3.5 weeks	01.02.2025	25.02.2025
Production	Program core mechanics	Movement, combat actions, special abilities	3 weeks	26.02.2025	18.03.2025
Prototype	Create prototype	Playable game where characters can move, attack the opponent, and determine a winner or loser.			
Production	Create assets	Characters, background, UI, level design, sound & music	8 weeks	19.03.2025	22.04.2025
Alpha	Alpha testing & bug fixing	Feature complete			
Beta	Balancing	Content complete	4 weeks	23.04.2025	20.05.2025
Launch	Final polish		2 weeks	21.05.2025	04.06.2025

Game genre and theme

Mythos grant is a fast-paced, 1v1 fighting game that blends dynamic platforming with strategic sword combat. The theme is inspired by an academic setting mixed with mythological stories. The stages and music will reflect this theme, blending modern and ancient aesthetics. The soundtrack will combine traditional instruments with electronic beats.

Game Mechanics, player goal, features

Players control characters who can move, jump, and dash around the screen while wielding large (proportionally sized) swords to attack their opponent. Each player has a health bar; when it is depleted, they lose the game. The goal of each player is to deplete the opponent's health before their own. The swords will be able to slash (rotation), stab (direct attack) and parry (timed swing or button push). Each sword user will also have a special ability that affects how they approach their opponent. The abilities are mostly statistical through buffs or debuffs. Each character will therefore have different attack, movement and defensive stats. Examples of abilities include slowing the opponent on a successful hit, extending the duration of a hitbox, and healing upon attacking an opponent. The game will feature local multiplayer, requiring players to either share a keyboard (using separate key bindings) or use an external controller.

Key technical elements

The game will be programmed in Godot v4.3, with art, sound, and music either AI-generated or sourced from online assets.

Art style

The game features a cartoonish art style. Characters have a distinctively limbless design, with their head, hands, and feet invisibly linked to their torso (inspired by Rayman). This design influences their movement and interactions with opponents' swords. Attire is inspired by the nationality of the sword each character uses. Stages—including backgrounds, walls, and platforms—are inspired by academic settings such as libraries, science labs, courtyards, cafeterias, PC labs, and auditoriums.

Story (World/Environment/Characters)

Players take control of characters armed with legendary sword relics, each imbued with unique abilities that reflect their historical significance.

Story: Historians reach an impasse over which ancient myth merits further research—and when scholarly debate falls short, there's only one way to settle it. In a one-off, high-stakes tournament, rival historians armed with legendary sword relics duel for supremacy. The winner's field of study secures the coveted grant money, ensuring their myth will be explored, and celebrated.

Environment: The arenas blend classical academia with mythic battlegrounds, ranging from ancient temples and historic cities to various rooms across a modern campus.

Character: Each character represents a different mythos, bringing their own research, pride, and unique sword-fighting techniques into battle.

Market positioning, target Platform, revenue and distribution model

Mythos Grant is positioned as a competitive 1v1 fighting game with fast, fluid movement inspired by Super Smash Bros. Ultimate, Super Smash Bros. Melee, and Celeste. It targets players who enjoy technical gameplay, precise movement, and strategic combat, offering a fresh twist by introducing mythological themes and unique interactions.

Platform: The game will initially be developed for PC (Windows) with local multiplayer support. Controller compatibility is a secondary priority but may be implemented to provide a familiar experience for fighting game enthusiasts.

Distribution: The plan is to release the game as a downloadable title on itch.io, with potential expansion to be determined.

Concept sketch

No concept sketch but plans are based on movement and interactions from Super smash ultimate, dash mechanics from Celeste, and a custom sword mechanic.

Milestone 2: Design

Design decisions

The game will be a "couch co-op" multiplayer game, allowing players to compete against each other on the same server or PC. While online multiplayer is a potential future feature, the primary focus is on local multiplayer to ensure optimal performance and minimal latency, which is crucial for the fast-paced gameplay.

Game Type:

Multiplayer: Primarily local ("couch co-op") with potential for online multiplayer in future updates.

Single-Player: Not a focus, but a training/ test mode could be added later.

Visual Style:

The game will be 2D or 2.5D, depending on the character models.

The art style will be low-poly, with animations created either through manual frame-by-frame animation or 3D rigging for 2D characters (2.5D).

Data Storage:

Data will be stored locally to minimize loading times and avoid lag spikes. Pre-loading assets will ensure smooth gameplay.

Platform:

Initially designed for PC, but with potential for console compatibility to broaden the player base.

User Interface (UI):

The UI will be minimalist, using simple image layering to maintain the game's aesthetic.

Features

The game's features are divided into must-have, should-have, and nice-to-have categories to prioritize development.

Core Gameplay Mechanics (Must-Have):

Movement:

Players can jump higher than the average platform height and dash in all 8 directions.

Dashing can transfer momentum when interacting with platforms or walls, adding depth to movement.

A single dash is available until the player lands or performs a wall jump.

There will be one button for both jump and dash.

Combat:

Players can swing or focus their sword.

The sword follows the player, either trailing behind or leading in front, depending on whether it's focused.

Sword swings gain power based on distance traveled but follow the shortest path to the target.

Parrying is possible by focusing the sword in front of the player. Successful parries stun the opponent, but parrying a swing causes the defender to lose to power.

Rock-Paper-Scissors Mechanics:

Stab beats Swing.

Parry beats Stab.

Swing beats Parry.

Special Abilities:

Each sword has a unique ability, such as:

Harpe: Slows the opponent's movement.

Tyrfing: Grants additional attack power but causes exhaustion afterward.

Nuada: Heals the user after multiple successful hits.

Zulficar: Performs a sweeping attack that can block enemy attacks.

Lævateinn: Confuses the opponent, randomizing their first three dashes.

Exploration and Progression (Should-Have):

Stages:

Each stage will have a unique background and platform setup.

The winner of each match influences the next stage, adding a dynamic element to gameplay.

Nice-to-have features include thematic elements (e.g., mythological references) and visual storytelling in the background.

Character Customization:

Players can earn in-game currency to unlock armor for their characters.

After 5 victories, a player's character becomes fully armored, symbolizing their dominance.

Accessibility and Difficulty (Must-Have):

The game's difficulty is player-driven, with no AI opponents or obstacles.

Character silhouettes will be distinct to ensure visibility, even for players with visual impairments.

Voice Lines (Nice-to-Have):

Characters will have voice lines when entering the stage, adding personality and immersion.

Example: A character might say,

"Let's make history—by erasing yours."

"I'd cite my sources, but you won't live to read them."

"Your thesis? Flawed. Your methods? Laughable. Your fate? Sealed."

"This blade has seen centuries. You won't last minutes."

"They say 'sword name' is unbreakable. Shame you're not."

Asset list

The game will require a variety of assets, including graphics, sound, and UI elements.

Graphics:

Character Models: Low-poly designs, either 2D or 2.5D.

Backgrounds: Unique environments for each stage.

Platforms and Walls: Customizable designs to match the stage themes.

UI Elements: Health bars, menus, and buttons.

Weapon Models: Unique designs for each sword.

Sound:

Background Music: Upbeat electronic music created using AI tools.

Sound Effects: Sword clashes, player damage, dashing, jumping, and stage transitions.

Voice Lines: Short phrases for character entrances and victories.

Other Assets:

Fonts: Easy-to-read, minimalist fonts.

Particle Effects: Sparks for sword clashes, dust for jumps and dashes.

Workflow:

Core Mechanics:

Default character, sword, stage, and platform models.

Multiplayer Integration:

Implement local multiplayer functionality

Immersion Mechanics:

Health UI, music, sound effects, and particle effects.

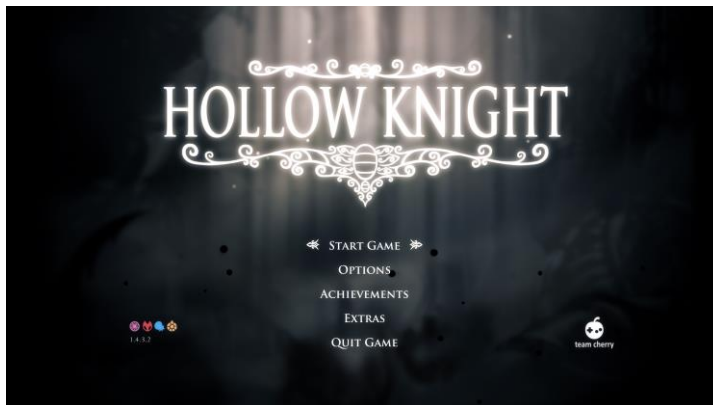
Additional Mechanics:

Menu screens, 8 sword models, 3 stages, and 8 wall designs.

Visual overflow

Scene Descriptions:

Main Menu: A simple, intuitive interface with options for Play, Controls, Settings, and Credits.



Like this only with thematic elements corresponding to the myths featured in the game.

Gameplay: Fast-paced combat on dynamic stages with distinct themes.



I want the platform layout to be inspired by *Celeste's* level design, where the foreground and background blend seamlessly to create a cohesive feel. However, the stages need to be much more open. I imagine some platforms and walls in the middle to break up the space and add more variation.

Victory Screen: A celebratory screen showcasing the winning player's character.
To be decided

Select Screen: Players choose their sword, stage, and music.



I want the character select screen to resemble this, but instead of characters, players will select swords (which correspond to characters). Both the swords and controls will be displayed on the sides, while common options like stage and music will be centered.

Control Screen: Customize controls for keyboard and controller.
To be decided

Settings: Adjust screen mode, music, and sound settings.
To be decided

User Flow:

Players start at the Main Menu.

They select Play and choose their characters and stage (music is set to default unless changed).

After the match, they return to the Main Menu or proceed to the next stage.

Art Style:

The low-poly aesthetic ensures a clean, modern look while keeping performance optimal.

Animations:

Character Movements: Smooth transitions for jumps, dashes, and attacks.

Environmental Effects: Subtle animations like moving clouds or flickering lights in the background.

Revenue plan

Monetization Strategies:

Self-Publishing: Release the game at a low cost and focus on marketing through social media and influencers.

Publisher Partnership: Pitch the game to a publisher for additional funding.

Post-Launch Content: Update with new stages, characters, or swords.

Pricing Model:

The game will be free at launch for alpha users to provide feedback, then increased to an adaptable price based on interest.

Marketing Plan:

Social media: Promote the game on platforms like Twitter, Instagram, and TikTok.

Influencers: Partner with gaming YouTubers and Twitch streamers.

Scope and time estimate

Development Phases:

Training

- Task: Learn new tools (e.g., Blender, Godot).
 - Duration: 2 weeks (integrated into Production phase).
 - Start Date: 26.02.2025.
 - End Date: 11.03.2025.

Asset Creation

- Characters and Swords:
 - Duration: 1 month (integrated into Production phase).
 - Start Date: 19.03.2025.
 - End Date: 18.04.2025.
- Stages and Backgrounds:
 - Duration: 1.5 months (integrated into Production phase).
 - Start Date: 19.03.2025.

- End Date: 22.04.2025.
 - UI and Sound:
 - Duration: 1 month (integrated into Production phase).
 - Start Date: 19.03.2025.
 - End Date: 22.04.2025.
-

Coding

- Core Mechanics:
 - Duration: 2 months (integrated into Production phase).
 - Start Date: 26.02.2025.
 - End Date: 22.04.2025.
- Additional Features:
 - Duration: 1.5 months (integrated into Production phase).
 - Start Date: 26.02.2025.
 - End Date: 22.04.2025.

Technical design

Game Engine:

Godot: Chosen for its flexibility, cross-platform support, and ease of use.

Software Tools:

Blender: For 3D modeling.

GIMP: For textures and UI design.

Audacity: For sound editing.

ComfyUI/Fugatto: For AI-generated assets.

Performance Optimization:

Reduce polygon counts for models.

Optimize code to minimize resource usage.

Risks

[Are there any risks in your project, e.g., what may cause you to run behind schedule]

Technical Risks:

Performance bottlenecks on lower-end hardware.

Networking issues for online multiplayer (if added).

Schedule Risks:

Delays in asset creation or coding.

Budget Risks:

Unexpected costs for tools or assets.

Market Risks:

Competition from similar games.

Changing player preferences.

Mitigation Strategies:

Regular testing to catch issues early.

Flexible timelines to accommodate delays.

Contingency budget for unexpected expenses.

Milestone 3: Prototype

Prototype implementation

Core Systems and Mechanics

During the prototype phase, the primary objective was to implement and test the fundamental gameplay systems in a controlled environment. This included movement, basic combat interactions, and local multiplayer functionality.

The implemented movement system allows characters to walk left and right, jump, and dash in eight directions. Dashes were designed to carry momentum, particularly in ground interactions, contributing to the intended fast-paced movement style. Early combat mechanics were also introduced, including basic sword swings. Although the attack system remained rudimentary at this stage, it served as a foundation for later expansion.

Local multiplayer functionality was established, allowing two players to engage using either a shared keyboard or a combination of keyboard and controller. This setup enabled consistent testing of interactions between players in real-time, an essential aspect of the game's intended 1v1 design.

Assets and Visual Design

All visual elements in the prototype were placeholder assets intended for functional validation rather than final presentation. These included:

- Characters: Default low-poly, limbless models were used, aligned with the intended minimalist aesthetic.



- Swords: A basic sword model was included for initial testing of interactions and orientation logic.



- Stage: A single test arena inspired by Super Smash Bros. Ultimate provided a neutral testing environment for platforming and combat.
- UI: Debug-level UI elements were included to display state and frame data relevant to testing.

These assets allowed for early assessment of visual clarity and gameplay readability, even without finalized artwork or animation.

Technical Implementation

The prototype was developed using Godot Engine version 4.3. Key technical implementations included:

- Physics: Custom physics code governed player movement, dash mechanics, and early sword interactions. This allowed for precise tuning of motion behavior.
- Input Handling: A modular input system enabled seamless mapping of keyboard and controller inputs, supporting two-player local gameplay.

- **Engine Features:** Core features of Godot, such as its 2D physics environment, scene graph, and animation tools, were evaluated for compatibility with the game's vision.

Prototype goals

The prototype aimed to verify the technical and design feasibility of the game concept. The key goals were:

- **Core Gameplay:** Establish basic movement, attack, and multiplayer systems.
- **Technical Feasibility:** Evaluate Godot's suitability for physics-based combat and input handling.
- **Player Interaction:** Ensure responsive and intuitive controls for two players in a local environment.
- **Visual Style Validation:** Confirm that a limbless, low-poly character design worked effectively in a 2D space.
- **Playability:** Enable basic matches using near-final control and movement logic.

These goals were largely achieved. The game was playable with local multiplayer support and demonstrated functional movement and early combat systems. Although not feature-complete, the prototype established a stable platform for further development.

Game state

Current Functionality

- Characters can move, jump, and dash.
- Local multiplayer with dual input support is functional.
- Basic sword swing mechanics are present.
- One test stage is implemented.
- Debug UI provides state feedback.

Missing Features

- **Combat Mechanics:** Stabbing, parrying, and special abilities remain unimplemented.
- **Stages:** Only one test stage exists; no themed or varied environments are present.
- **UI/UX:** Menus, victory screens, and player instructions are missing.
- **Sound and Music:** No audio elements are currently included.
- **Game Balance:** Variables such as movement speed, dash power, and attack damage require tuning.

What did I learn

Technical Insights

- The Godot engine's physics system is well-suited to 2D platforming and momentum-driven mechanics.
- Implementing local multiplayer was straightforward but required careful input mapping and consistent device detection.

Design Considerations

- Combat pacing is closely tied to movement responsiveness; careful balancing is necessary to maintain game flow.
- The limbless, low-poly character design performed well in gameplay, though it introduces animation challenges that must be addressed creatively.

Next Steps

- Implement advanced combat mechanics, including stabbing, parrying, and character-specific abilities.
- Design and build additional stages with varied platform layouts and visual themes.
- Develop user interface elements for match flow, settings, and player feedback.
- Integrate sound effects and background music to enhance game feel and provide auditory cues.

Prototype Progress Summary		
Feature	Status	Notes
Movement (Jump, Dash)	Implemented	Momentum transfer during dashes needs fine-tuning.
Basic Sword Swings	In the works	Sword rotation needs fixing.
Multiplayer (Local)	Implemented	Keyboard and controller inputs are mapped.
Stabbing and Parrying	Not Implemented	Core combat mechanics still in development.
Special Abilities	Not Implemented	Planned for future updates.
Stages	One test Stage	Additional stages with unique themes needed.
UI/UX	Not Implemented	Requires thematic polish and additional features (e.g., settings menu).
Sound and Music	Not Implemented	Background music and sound effects are pending.

Milestone 4: Alpha

Alpha Implementation

Core Systems and Mechanics

The focus of the Alpha milestone was to establish the foundational combat mechanics necessary to begin evaluating character responsiveness and interaction. Development proceeded from the core outward, prioritizing player movement and weapon systems before introducing auxiliary components such as health tracking, menus, or audio. The primary goals included implementing player-controlled movement, a functional pendulum-based sword mechanic, and the beginnings of a combat feedback loop.

Movement

Character movement was implemented via a physics-based system in which player motion is driven by forces and velocities rather than animation. Input can be received from both keyboard and controller devices. Initially, interactions between characters were tested using body collisions, before transitioning to a system where a sword applies knockback when it connects with an opponent. Knockback was made directionally responsive by calculating the relative horizontal position between the attacker and defender, enhancing the intuitive feel of the combat system.

Sword mechanics

The pendulum sword mechanic serves as both a visual element and the core weapon system. It trails behind the player character and features a constant hitbox. To enable offensive intent, an input-triggered mechanic was introduced that increases the sword's angular momentum, allowing players to direct its movement toward opponents. Hitlag and hitfreeze were added to further support readability and impact; upon contact, game motion is momentarily paused to emphasize successful strikes.

Fine-tuning physics

Refining the pendulum motion required substantial effort. Tuning responsiveness involved adjusting damping, pivot delay, and swing momentum. A key technical challenge emerged due to erratic sword behavior during high-speed movement, caused by limitations in Godot's built-in physics interpolation. This was resolved by implementing a custom interpolation method, which ensured consistent sword behavior and preserved the desired sense of momentum.

Input and Control System

Input support was built modularly, allowing keyboard and controller devices to operate interchangeably. Controller integration was straightforward, involving the mapping of controller inputs to existing player actions without major codebase changes. This modular approach will also facilitate future input extensions.

Directional knockback and player control systems were tested across both input types. These experiments provided early insight into tuning player responsiveness, ensuring that actions such as attacking and dashing felt equally precise with both input modes.

Visual Feedback and Combat Cues

Combat responsiveness was enhanced through the addition of several visual effects. The sword's motion, already visually prominent due to its pendulum behavior, was emphasized

further by brief pauses (hitfreeze) and timing adjustments (hitlag) during strikes. These enhancements helped create a more readable and impactful combat experience, even in the absence of traditional character animations.

Although visual polish was minimal at this stage, the sword's movement, dynamic knockback reactions, and hit pauses collectively provided the player with immediate feedback, laying the groundwork for more complex audiovisual responses in later milestones.

Alpha Goals

The primary objective during Alpha was to validate the core concept of physics-driven, pendulum-based combat between two player-controlled characters. By the end of the milestone, this goal had been achieved. Players could move, dash, jump with variable height, swing swords, and react to being hit. Knockback scaled with accumulated damage, and basic hit detection logic was in place.

While the prototype did not yet constitute a complete game, it demonstrated that the central gameplay loop—centered on positioning, movement, and directional attacks—was viable and responsive.

Game State

At the conclusion of the Alpha milestone, the game existed as a mechanical prototype with the following features:

Character movement with dashing and variable jump height

A pendulum-based sword system with a dynamic hitbox

Damage-scaling directional knockback

Visual feedback via hitfreeze and hitlag

Simultaneous play for two players using either keyboard or controller

The following components had not yet been implemented:

Health system and win/loss conditions

Menus, UI elements, or player instructions

Sound effects and background music

Environmental visuals such as backgrounds or screen transitions

Wall jumping and advanced traversal mechanics

Particle effects and polish-level feedback systems

Additionally, the original parry mechanic was replaced by a system where players could knock away the opponent's sword using momentum. The stage geometry had been created,

but no background visuals were included; these were deferred to the Beta phase. Most development work at this stage prioritized functionality over code clarity or optimization.

What Did I Learn

This phase of development highlighted the importance of early and frequent iteration, particularly when working with real-time physics and non-traditional weapon systems. Small adjustments to physics values—such as damping coefficients or swing thresholds—had a substantial impact on gameplay feel. The project also demonstrated that traditional animation is not strictly necessary to convey responsive action, provided that timing and feedback are well-tuned.

Developing a consistent and expressive sword system required custom interpolation logic, providing valuable experience with motion tracking and physics behavior in the Godot engine. Seemingly simple systems, such as knockback direction or hit detection, involved considerable debugging due to the complexity of real-time interactions.

Regular playtesting proved indispensable. Observing character behavior under varying input and collision conditions offered clear guidance on what aspects needed refinement. The continuous test-adjust-repeat cycle was essential in shaping the mechanical identity of the game during this early stage.

Milestone 5: Beta

Beta implementation

Core Systems and Mechanics

This milestone focused on extending and refining the foundational systems implemented during Alpha to bring the experience closer to a complete game. A key enhancement was the implementation of a visible health system. The health bar sprite was designed in Piskel, with visual inspiration drawn from Canva-style assets. Whereas a hidden damage value had previously been incremented upon successful hits, a segmented health bar was now added. This visual indicator dynamically changes color based on remaining health and integrates with the knockback system. When a player's health is fully depleted, the round ends, although full round-to-round continuation logic is still pending. The current knockback response, which scales with health, was identified as overly strong and will require balancing in future iterations, possibly by introducing a dampened or static scaling model.

Player movement was also improved. Wall jumping was introduced, expanding the player's mobility options. In addition, a dash-input buffer was added, enabling a more forgiving input window. Players may now queue a dash or jump slightly beyond the termination of a dash, resulting in smoother and more accessible movement sequences.

User Interface and Navigation

A functioning menu system was added, including transitions between the title screen and gameplay scenes. Significant effort was directed toward ensuring reliable controller and keyboard navigation. The project encountered challenges regarding focus behavior, particularly when navigating menu options using the D-pad or directional keys. To address this, focus modes were fine-tuned and parts of the button interaction system were rewritten. These improvements ensure that input devices consistently highlight and activate the correct menu elements.

Audio Implementation

Audio feedback was introduced using `AudioStreamPlayer` nodes. This includes background music and a range of sound effects tied to core player actions such as attacks, successful hits, jumping, and landing. Careful attention was given to volume balancing, ensuring that music remained supportive without overpowering essential sound cues. Each sound was selected to provide clear, distinct feedback and contribute to the overall responsiveness of the combat system.

Visual Feedback and Effects

To enhance the tactile feel of combat, several visual effects were introduced. A screen shake effect was implemented to emphasize impactful moments, particularly upon landing attacks. In addition, a hit flash overlays a brief color cue on characters when struck, reinforcing the connection between cause and effect during engagements. These feedback layers contribute to a more dynamic and satisfying gameplay experience.

Beta Goals

The primary goal for the Beta milestone was to solidify the foundational combat loop. This included establishing health tracking, knockback dynamics, visual and audio feedback, and basic UI flow. Although full win-state logic has not yet been implemented, the current structure allows for coherent round resolution. Special attention was given to refining the

timing and responsiveness of player inputs and layering audiovisual elements so that actions such as dashing, jumping, slashing, and hitting feel distinct and impactful.

Game State

By the conclusion of the Beta phase, the game included a robust and playable combat loop consisting of:

Character movement including running, dashing, jumping, and wall jumping

A pendulum-based sword attack system

Health bars integrated with knockback and round-end logic

Directional knockback and hitstop (pause) feedback

Screen shake and hit flash effects

Layered audio including background music and sound effects

A semi-functional main menu with scene transitions to gameplay

A background image to visually anchor the stage

At this stage, the game was structurally complete in its core mechanics but still required additional features and polish. Remaining tasks audio settings toggles, visual design for the victory screen, improved particle effects, and a final pass over assets for sound and art consistency. Full animation support has not yet been implemented and has been down prioritized for later implementations.

Milestone 6: Live

Final Implementation

Following the Beta version, development efforts focused on polish, user experience improvements, and bug fixes informed by playtesting.

Victory Flow

To complete the core game loop, a scoreboard UI was implemented to track round victories and losses. The scoreboard sprite was designed using containers and multiple joined polygons. A Victory Screen UI was introduced, displaying post-match outcomes and allowing players to either restart the match or return to the title screen. The round length can be adjusted through toggleable UI buttons.

Combat Tweaks

Several adjustments were made to combat based on tester feedback. Knockback previously lacked variety and appeared overly linear. Directional knockback was introduced, making hit reactions reflect the angle of impact—for example, upward strikes now lift opponents, while downward strikes push them toward the ground. To reduce chaotic exchanges in which both players could retaliate instantly, a brief hit buffer was added to prevent immediate counterattacks.

Additionally, a mechanic was added where attacking a sword in the "aim" state results in greater knockback to the defending player. This discourages overly aggressive charging tactics and adds strategic depth.

UI/UX Improvements

To address usability issues, an instruction screen was added between the title menu and gameplay to convey basic mechanics. While currently always shown, future iterations will allow toggling or hovering to minimize disruption. An Options menu was also created, enabling toggling fullscreen, adjusting music and sound volumes, and accessing a Credits screen listing development attribution.

To assist with player differentiation—especially since characters share similar base designs—color variations were added to sprites and animations. Additional polish included aligning the background with the camera to avoid visual displacements and increasing ground dash distance to better match the responsive feel of aerial movement.

Content Additions

A late-stage feature added narrative flavor: a health-regenerating item framed as research “funding.” When hit, players lose “funding”—represented by coin particle effects and a wallet sound. Upon defeat, the character's wallet drops and scatters coins, thematically reinforcing the game’s premise of fighting for academic grant money.

Technical Fixes

A major bug identified during Beta caused retained knockback velocity after respawn, leading to unintended post-respawn launches. This was resolved by isolating knockback logic and preventing its application during the respawn phase.

Music was rerouted through an audio bus to allow global audio toggle controls. The project was then packaged as an executable and published via GitHub and Itch.io, meeting distribution goals and ensuring version control.

Final Goals

Core Loop Completion

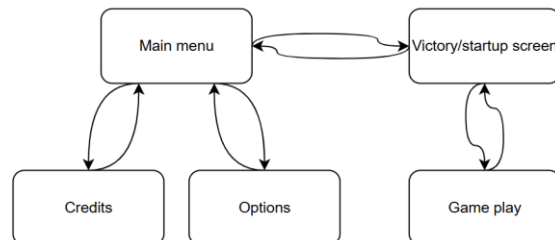
The game now includes a fully implemented gameplay loop: match initiation, dynamic combat, round resolution, and transitions to rematch or menu states.

Responsive Gameplay

Combat interactions have been refined to feel responsive and satisfying through features such as directional knockback, buffered hitstun handling, and screen shake feedback. The pendulum-based sword physics system now fulfills its original mechanical intentions.

UI and Navigation Readiness

All core UI elements are functional, including the Title Menu, Options, Instructions, Victory Screen, and Credits. Navigation across these screens is intuitive and does not require external guidance. Here's an illustration regarding how it works, which showcases how the player can move back and forth between each scene:



Playability and Accessibility

The game can be played and understood by new users without developer intervention, aided by visual identity for characters, instructional guidance, and refined controls.

Public Availability

The game has been released as a playable executable on Itch.io:

<https://rehunterg.itch.io/mythos-grant>,

with source code maintained through GitHub:

<https://github.com/JensVL96/Mythos-grant>

Game Evaluation

Game State

The game delivers a complete and engaging 1v1 combat experience, successfully achieving its core design objectives. The core player feedback loop—sword swinging, impact registration, knockback, and victory—is now consistent and satisfying.

Combat System

The pendulum-based sword system behaves predictably while maintaining depth through momentum and physics-based dynamics. Directional knockback and hitstun buffers create a dynamic but readable combat rhythm.

User Experience

Playtesters provided positive feedback, highlighting the game's unique mechanics and intuitive feel. Visual enhancements such as particle effects and color-coded characters contributed to a more engaging experience.

Game Flow

The full cycle of gameplay—from match start to victory resolution and return to the title screen—is complete and functions without known bugs. Round customization adds player agency over pacing.

Technical Stability

Critical bugs—such as unintended knockback persistence—have been resolved. The executable runs reliably on target devices, and restarts no longer inherit unintended physics states.

Known Limitations

Sword customization and full animation cycles were deferred due to time constraints. Character differentiation is currently based solely on color variation.

Animations remain minimal, reducing expressive potential during moments like spawn and defeat.

Controller support exists but was disabled by default in the live version. Only GameCube controllers were tested, limiting compatibility.

Asset theming is inconsistent; various elements were AI-generated from different sources, resulting in a mixed visual identity.

Future Work

Narrative & Lore Integration

Future updates will focus on integrating narrative elements more deeply with gameplay. Planned features include multiple character variants, unique swords with lore significance, and visual representations of player progression through armor acquisition. A background mechanic is also planned, wherein stages visually shift to reflect the cultural dominance of each sword's origin based on match performance. This would include dynamic particle-based transitions between background states.

Combat Enhancements

Improvements to sword collision detection will increase the perceived impact of clashes. Current collision-box interactions may result in overlap without strong feedback. Refinements will aim to make sword swings feel weighty and deliberate. Additionally, the dash system—while functional—will be expanded with mechanics inspired by Celeste, offering greater mobility depth and rewarding skillful execution.

Controller Integration

Although controller navigation is currently supported, full controller play was removed due to limited hardware testing. Broader support for various controller types and user toggling will be prioritized in future iterations.

Visual and Audio Consistency

Presently, assets are drawn from a mix of AI-generated and royalty-free sources. Future work will focus on developing a cohesive art direction, ensuring characters, environments, and UI components present a unified aesthetic.

Cost Breakdown

Category	Tools/ Assets used	Cost
Game engine	Godot	Free
Art tools	Piskel – pixel editor	
	GIMP – image editor	
	Sprite fusion – map editor	
Audio tools	Audacity	
AI-generated assets	Suno - music	
	ChatGPT sprites & background image	
Online assets	Pixabay - sounds	
	Kenney – UI borders	

Total Budget: Free

What did I learn?

Scope Management: Prioritized essential polish over secondary features such as detailed animations.

Iterative Design: Playtesting informed valuable improvements, especially in combat feel and UX.

Technical Debugging: Custom systems, particularly the pendulum sword mechanic, required ongoing troubleshooting and refinement.

User Experience: Exposure to real-time feedback led to improved accessibility and player onboarding.

Screenshots

This is the main menu, where the player is able to change sound, music and screen settings, or just jump straight into the game.



Once the player enters the game this is the first he'll see. This is the tutorial for mechanics in the game



Then the player is given this simple UI for making game decisions, for displaying the score in between each round and for tracking his health during the match



Lastly this is the game screen were the action happens:



Student Contribution

This project was developed independently, encompassing all aspects of design, implementation, testing, and deployment. While feedback was incorporated from peers and instructional staff, all code, assets, and polish were produced by the student.